

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115 Bug 15 Code Smell 100

TagsImpactClean code attributeSearch by name...

Code annotated as deprecated should not be used
Code Smell
File names should comply with a naming convention
Code Smell
Unused local variables should be removed
Code Smell
Package names should comply with a naming convention
Code Smell
Overriding methods should do more than simply call the same method in the super class
Code Smell
"isEmpty" or "isNotEmpty" should be used to test for emptiness
Code Smell
Unnecessary imports should be removed
Code Smell
Empty statements should be removed
Code Smell
Class names should comply with a naming convention
Code Smell
Track uses of "TODO" tags
Code Smell
Deprecated code should be removed
Code Smell
Cyclomatic Complexity of functions should not be too

Code annotated as deprecated should not be used

Analyze your code

Consistency - ConventionalMaintainabilityCode SmellMinorcwecwe obsolete

Why is this an issue?More Info

Code is sometimes annotated as deprecated by developers maintaining libraries or APIs to indicate that the method, class, or other programming element is no longer recommended for use. This is typically due to the introduction of a newer or more effective alternative. For example, when a better solution has been identified, or when the existing code presents potential errors or security risks.

Deprecation is a good practice because it helps to phase out obsolete code in a controlled manner, without breaking existing software that may still depend on it. It is a way to warn other developers not to use the deprecated element in new code, and to replace it in existing code when possible.

Deprecated classes, interfaces, and their members should not be used, inherited or extended because they will eventually be removed. The deprecation period allows you to make a smooth transition away from the aging, soon-to-be-retired technology.

Check the documentation or the deprecation message to understand why the code was deprecated and what the recommended alternative is.

```
@Deprecated("This function is deprecated, use newFunction instead", ReplaceWith("newFunction()"))
void oldFunction() {
    println("This is the old function.");
}

void newFunction() {
    println("This is the new function.");
}

oldFunction() // Noncompliant: "oldFunction is deprecated"
```

Exceptions

The rule does not raise an issue when the deprecated members are used in a deprecated type.

Available In:

sonarcloud

